

Knowing Where to Look Is Not Knowing What Is Right: A Difference-Aware Transformer for Physical Commonsense QA

Group 69

Abstract

PIQA asks which of two near-identical solutions achieves an everyday physical goal. We ask whether a model trained from scratch, without external data, can learn *where* the two solutions differ, and whether that is enough. Our Difference-Aware Contrastive Transformer (DACT) makes the difference explicit, with difference-tag embeddings, cross-solution attention and difference-guided pooling, and adds a jointly trained lexical head. The answer is yes to the first question and no to the second. Attention controls show that DACT learns to put 86% of its pooling attention on the differing words (uniform attention would give 25%), and ablations confirm that marking the difference helps. But knowing where to look is not knowing what is right: DACT is the best from-scratch model on validation (61.2%, 3 seeds), yet on test it is statistically tied with a TF-IDF baseline (61.6 vs. 61.2%, McNemar $p=0.88$), and pretrained models are 5–14 points better. Wrong predictions attend to the differing words even more than correct ones: the bottleneck is physical knowledge, not focus. We also report a pre-registered negative result and the failures that shaped the design.

1 Introduction

Physical commonsense, knowing that *baby wipes* rather than *index cards* remove chalk dust, is easy for people and hard for machines. PIQA (Bisk et al., 2020) tests it with a goal and two candidate solutions, exactly one of which is physically sensible. The task matters for assistants that give practical instructions and for robots that must choose feasible actions. Its key difficulty is that the two solutions are usually near-duplicates: in our training split the median share of differing solution tokens is 15%, so the answer hinges on a few substituted words whose plausibility requires world knowledge.

Related work. On PIQA, pretrained models are far from people: Bisk et al. (2020) report 77.1% for RoBERTa-large against 94.9% for humans, and multi-format QA training improves pretrained models further (Khashabi et al., 2020). Our setting differs, because the main model must learn from scratch. The closest architectural ideas come from multiple-choice reading comprehension and inference. The Option Comparison Network (Ran et al., 2019) and DCMN+ (Zhang et al., 2020) compare candidate answers with each other instead of scoring them independently, and ESIM (Chen et al., 2017) and the decomposable attention model (Parikh et al., 2016) align two texts token by token and build comparison features. DACT adapts these ideas to two near-identical solutions: difference tags and cross-solution attention make the alignment explicit, and difference-guided pooling focuses the decision on the substituted words. Finally, work on annotation artefacts shows that shallow lexical cues can solve many items without reasoning (Gururangan et al., 2018; Poliak et al., 2018); our TF-IDF baseline and lexical head make this effect measurable on PIQA.

Our aim was a model that is trained from scratch but built around this structure. DACT (Section 2) makes the difference between the solutions explicit at every stage: in the input, in the comparison and in the summary vector, and it adds a lexical head that scores which words make a solution plausible. We compare it with five baselines, from majority vote to a zero-shot LLM, run eight ablations with three seeds each, and analyse the attention with controls and probes.

Our main findings are: (i) a from-scratch model does learn where the options differ: the attention focus on the differing words is learned, not built in, and removing the difference tags lowers accuracy; (ii) this is not enough: DACT is the best from-scratch model on validation but statistically tied with a strong lexical baseline on test; (iii) errors

come from *judging* the differing words without physical knowledge, which only pretraining supplies. Section 5 documents what worked and what did not during development.

2 Methodology

Input and difference tags. For a goal g and solutions s_1, s_2 , each option k is encoded as $x^{(k)} = [\text{CLS}] g [\text{SEP}] s_k [\text{SEP}]$ with a BPE vocabulary (Sennrich et al., 2016) learned on the training split. We align the token sequences of s_1 and s_2 with `diff1ib` and give every solution token a tag $t_i \in \{\text{shared}, \text{different}\}$ (goal and special tokens: *other*). The input embedding is

$$e_i = E_{\text{tok}}[x_i] + E_{\text{pos}}[i] + E_{\text{seg}}[\sigma_i] + E_{\text{tag}}[t_i]. \quad (1)$$

The tag embedding (our contribution) tells the encoder from the first layer where the options disagree.

Encoder. A shared pre-LayerNorm Transformer encoder (Vaswani et al., 2017; Xiong et al., 2020) maps each option to hidden states $H^{(k)} \in \mathbb{R}^{L \times d}$.

Contrastive cross-solution attention. Each token of option k attends to the solution tokens of the rival option $\bar{k}$ plus its CLS token as a “no counterpart” sink:

$$c_i = \text{MHA}(h_i^{(k)}, H^{(\bar{k})}, H^{(\bar{k})}), \quad (2)$$

and the states are updated with the comparison features of ESIM (Chen et al., 2017): $\tilde{h}_i = \text{LN}(h_i + \text{FFN}([h_i; c_i; h_i - c_i; h_i \odot c_i]))$. Adapting ESIM’s local inference from premise–hypothesis pairs to two competing answers is our design choice: plausibility is relative, so each option is represented by what it has *instead of* its rival.

Difference-guided pooling. Over the solution tokens we use additive attention (Bahdanau et al., 2015) with a learned bias per tag (our extension):

$$\alpha_i \propto \exp(v^\top \tanh(W\tilde{h}_i) + b_{t_i}), \quad p = \sum_i \alpha_i \tilde{h}_i, \quad (3)$$

with $b_{\text{different}}$ initialised to 1 and the others to 0. A scorer MLP maps $[p; \tilde{h}_{\text{CLS}}]$ to a scalar s_k .

Lexical head. Following the wide & deep idea (Cheng et al., 2016), a sparse linear component scores the words of each solution directly: every BPE unigram and bigram of s_k is hashed into one

of 2^{18} buckets with learned weight w_j (initialised to 0),

$$\ell_k = \sum_{j \in \text{ngrams}(s_k)} w_{h(j)}, \quad z_k = s_k + \ell_k. \quad (4)$$

It is trained jointly with the Transformer, with its own learning rate and weight decay. Under the pairwise softmax only $\ell_1 - \ell_2$ matters, so shared n-grams cancel and the head learns which substituted words indicate a plausible solution, the signal that a TF-IDF baseline exploits.

Objective. The two scores compete in a softmax, $P(k) = \text{softmax}(z_1, z_2)_k$, trained with cross-entropy and label smoothing 0.1, which matches the forced-choice task. Every component can be switched off in a single configuration object, which is how the ablations are run.

3 Experimental Setup

Data. PIQA provides 16,113 training items and 1,838 labelled development items; the latter serve as our test set. Test labels are used only in the final evaluation step (enforced by a guard in the notebook), although the test file is loaded and tokenised at the start and B1/B3 compute (but do not inspect) their test predictions earlier. An audit of the data found six duplicate rows within the training file and one test item (index 1545) that appears twice in our training split. We keep the split unchanged and disclose the overlap: re-scoring the saved test predictions without that item changes accuracy by about 0.02 points (DACT $61.57 \rightarrow 61.55\%$, TF-IDF $61.21 \rightarrow 61.19\%$). We hold out a stratified 10% of the training file (seed 42) for validation: 14,501 / 1,612 / 1,838 items, all label-balanced (50.0 / 50.0 / 50.5% label 1). Text is whitespace-normalised, NFKC-normalised and lower-cased. Goals are truncated to 40 and solutions to 96 BPE tokens; for longer solutions (1.6% of test solutions) we keep the 96-token window centred on the differing span, not the prefix (Section 5).

Training. AdamW (Loshchilov and Hutter, 2019) ($\beta_2=0.98$, weight decay 0.05, learning rate 3×10^{-4} ; lexical head 10^{-2}), 6% linear warm-up then cosine decay, batch 128, gradient clipping 1.0, dropout 0.2, at most 20 epochs with early stopping on validation accuracy (patience 6), bf16 mixed precision on a Colab A100. Step 1 of the notebook selects the configuration on validation with one seed from five candidates: base ($d=256$,

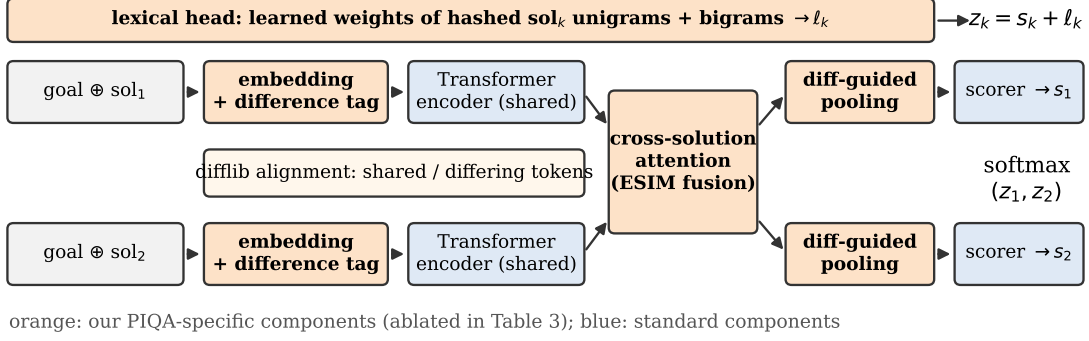


Figure 1: DACT. Both options pass through the same weights, so for already-encoded inputs swapping the options swaps the scores (verified in the notebook). The full pipeline is not exactly equivariant, because the diffliab alignment depends on option order (Section 5).

4 layers) 61.0%, **small** ($d=128$, **2 layers, 4 heads**) **61.3%**, base with dropout 0.3 60.9%, and base/small with a 10-epoch in-domain masked-LM warm-up (Devlin et al., 2019) 60.9 / 61.0%. The selected model has 1.93 M parameters, 262 k of them in the lexical head. The lexical head’s learning rate was chosen in a separate validation pilot (Section 5).

Baselines. All baseline numbers come from our own runs. **B0** majority class. **B1** TF-IDF of 1–2-grams of $s_2 - s_1$ with logistic regression, C tuned on validation; it tests how far shallow lexical cues go (Gururangan et al., 2018). **B2** a 2-layer BiLSTM with additive attention and the same tokenizer (5.1 M parameters), a recurrent model of similar budget. **B3** roberta-base (Liu et al., 2019) fine-tuned with a multiple-choice head (4 epochs, learning rate 2×10^{-5} , 3 seeds, best epoch on validation), representing pretrained encoders. **B4** Qwen2.5-1.5B (Qwen Team, 2024) zero-shot: the prompt “Goal: g / Solution:” followed by each solution, choosing the higher length-normalised log-likelihood, which measures LLM knowledge without any training.

Metrics. Accuracy is PIQA’s official metric; labels are balanced and each item has exactly one correct option, so it is unbiased with 50% chance level. Trained models are run with seeds 42–44 and reported as mean $\pm$ std. For the seed with the best *validation* accuracy we give a 95% bootstrap CI (2,000 resamples; Efron and Tibshirani, 1993) and an exact McNemar test (McNemar, 1947) against DACT on the test set.

Model	Val	Test
<i>trained from scratch</i>		
B0 majority	50.0	50.5
B1 TF-IDF + LR	59.2	61.2
B2 BiLSTM + attention	59.5 ± 1.2	60.1 ± 0.7
DACT (ours)	61.2 ± 0.4	61.6 ± 0.1
<i>pretrained</i>		
B3 RoBERTa-base, fine-tuned	68.0 ± 1.9	66.7 ± 1.1
B4 Qwen2.5-1.5B, zero-shot	78.1	75.4

Table 1: Accuracy (%), mean $\pm$ std over 3 seeds; B0, B1 and B4 are deterministic single runs. DACT test 95% CI: 59.4–63.6.

Validity of the test numbers. Validation accuracy is the only number that reflects design decisions. The test set was evaluated in several development runs, and the lexical head was introduced after earlier test results had been seen (Section 5). Test numbers are therefore reported for completeness and comparison, not as a clean held-out estimate of generalisation.

4 Results and Analysis

Overall comparison. DACT is the best from-scratch model on both splits (Table 1): +1.7 / +1.5 points over the BiLSTM (not significant on test, McNemar $p=0.10$) and +2.0 / +0.4 over TF-IDF. On test, DACT and TF-IDF are statistically tied (61.6 vs. 61.2%, McNemar $p=0.88$): DACT’s advantage over the lexical baseline is on validation, and the error analysis below shows where each model wins. We do not claim that DACT beats it. Pretraining dominates: RoBERTa is 5.1 and the zero-shot LLM 13.8 points above DACT on test, in line with PIQA being designed to test knowledge rather than reading skill (Bisk et al., 2020).

Ablation (vs. full DACT)	Δ Val	Δ Test
– difference tags	–1.3	–1.0
– lexical head	–1.8	–0.4
pointwise (independent BCE) objective	–0.7	0.0
mean instead of attention pooling	–0.1	–0.6
– cross-solution attention	–0.7	–0.9
– tag bias in pooling	+0.5	+0.2
tag bias initialised at 0	+0.4	+0.1
vanilla (all of the above removed)	–2.5	–2.4

Table 2: Ablations, 3 seeds each, one change at a time on the selected configuration; differences of the 3-seed mean accuracies, each rounded to 0.1 points before subtracting (seed std $\approx$ 0.3–1.2).

Ablations. Each ablation tests one hypothesis about a design choice (Table 2); everything else, including seeds, data and budget, is fixed. *Marking and comparing the difference helps*: removing the tags costs 1.3 / 1.0 points and removing cross-solution attention 0.7 / 0.9. *Direct lexical evidence helps*, mainly on validation: removing the lexical head costs 1.8 / 0.4. The forced-choice objective helps on validation only (independent binary scoring: 0.7 / 0.0). Removing everything costs 2.5 / 2.4 points, the largest drop. Attention pooling makes little difference (0.1 / 0.6), and the tag bias, including its initial value, makes no consistent difference (removing it gives +0.5 / +0.2): these components overlap with the tags, which already tell the model where the options differ. Differences below one seed std are not conclusive, and with seven comparisons single p-values should not be over-read (Dror et al., 2018).

Complementary errors. DACT and TF-IDF agree on 77.9% of test items; DACT alone solves 11.2% (205 items) and TF-IDF alone 10.9%, for an oracle accuracy of 72.4%. The items only DACT solves typically have a one-word difference inside long shared text (*water* vs. *soda*; the typo *sparrow bottle* vs. *spray bottle*), where a bag-of-words difference vector has almost nothing to work with. Against RoBERTa the oracle rises to 83.6%.

Is the attention focus learned? The trained model puts 85.9% of its pooling attention on differing tokens, which make up 24.8% of solution tokens (Figure 2). Because the tag bias starts at 1, we separate the sources: an untrained model reaches only 38.9%, while the trained model with the bias set to 0 still reaches 75.1%, and the bias itself barely moved in training (1.0 $\rightarrow$ 1.019). The

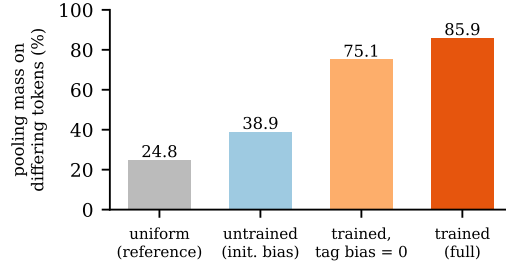


Figure 2: Share of pooling attention on differing tokens (test set). Removing the tag bias from the trained model keeps most of the focus, so it is learned, not built in.

focus is therefore learned by the pooling scorer, which also explains why initialising the bias at 0 changes nothing. Attention weights are not a complete explanation of a decision (Jain and Wallace, 2019; Wiegrefe and Pinter, 2019), so we use them as evidence of where the model looks, not of why it decides.

Why does the neural part alone not beat bag-of-words? We tested two explanations on validation. *H1: the pooling does not select* is rejected: the normalised pooling entropy is 0.49 (1 = uniform), for correct and wrong predictions alike ($p=0.18$). *H2: the difference signal is missing*: a linear probe separating differing from shared tokens reaches 59.0% balanced accuracy on the vanilla Transformer’s embeddings and only 59.9% after its encoder, so without tags the encoder barely works out where the options differ, whereas in DACT the tag remains perfectly decodable (100%) in every layer. Tags and the lexical head thus supply information the encoder does not learn from 14.5 k items. Crucially, wrong predictions put *more* attention on the differing tokens than correct ones (87.7 vs. 84.7%, $p<0.001$): looking at the right words is not the bottleneck; judging them is.

Qualitative analysis. For the item *adding scents to lotion bars* (Figure 3, top), the wrong option adds the oils “to the water in the bottom of the double boiler”. The pooling peaks on *bottom* (of the double boiler), the part that makes the option wrong, yet the model prefers it with $p = 0.99$: it locates the decisive span but cannot know that oil in the water bath never reaches the lotion. In *to safely sleep with your baby* (bottom), pooling selects the differing spans on both sides, but the scores tie: the choice needs infant-safety knowledge. Successes rest on the same word-level evidence: for

Goal: Adding scents to lotion bars

Option 1, $p = 0.99$ grate the wax and melt, in a double boiler. add the oils to the water in the bottom of the double boiler.

Option 2 (gold), $p = 0.01$ once the lotion and wax have melted, and reached the trace stage, add the scented oils and stir to combine.

Goal: To safely sleep with your baby in your bed

Option 1, $p = 0.50$ lay under a light cover only, keep yourself between them and the other parent and put pillows on the ground by the bed in case the baby falls off.

Option 2 (gold), $p = 0.50$ lay on top of the cover, keep yourself between them and the other parent or preferably have the other parent sleep somewhere else, and put up a barrier to keep the baby from falling onto the floor.

Figure 3: Pooling attention over each solution (darker = higher weight; **bold** = tokens that differ between the two options). Top: *adding scents to lotion bars*, a confident error ($p = 0.99$ for option 1). Bottom: *to safely sleep with your baby in your bed*, the most uncertain item ($p = 0.50$; option 2 is gold).

a simple camping lantern, the gold option (a milk jug on top of an *LED light, translucent plastic*) wins with $p = 1.00$ against *fairy lights* in a filled jug, with pooling spread over the many differing words. In an earlier run, *baby wipes* vs. *index cards* for chalk dust was solved confidently; the training split contains *baby wipe(s)* only in the correct option 26 times and only in the wrong option 5 times, so the confidence comes from word statistics, not from reasoning about absorbency. Finally, the most confident error, *boil water for a cup of tea* ($p = 0.99$), shows the lexical head’s calibration problem: the wrong option adds a whole extra sentence, and since ℓ_k sums over n-grams, long differing spans receive extreme scores.

5 What Worked and What Did Not

Table 3 summarises how the final model came about. All runs are logged in the repository; only the final A100 run of the submitted code is used in Tables 1–2.

What worked. Difference-aware truncation removed inputs that had become identical after prefix truncation (in run 1, one item differed only at BPE positions 97–98). The attention controls (Figure 2) turned the worry that the focus was built in by the bias initialisation into a test, which it passed. Three RoBERTa seeds exposed fine-tuning instability: in run 2 one seed stayed at chance (53.7% validation), a known failure (Dodge et al., 2020). Giving the lexical head its own learning rate made it useful.

Step	Change	Val / Test	Outcome
Run 1 (A100)	original design, prefix truncation	60.1 / 60.4	below TF-IDF on test
Run 2 (T4)	difference-aware truncation, attention controls	59.4 / 60.2	focus is learned
Pilot (val)	lexical head: shared → own learning rate	59.5 → 60.9	own rate needed
Run 3 (T4)	lexical head	61.1 / 61.6	above TF-IDF on val
Final (A100)	submitted code	61.2 / 61.6	reproduces run 3
Pre-reg. (val)	goal-matching block	60.9 vs. 61.3	rejected, no test run

Table 3: Development history: DACT accuracy (%). TF-IDF + LR scores 59.2 / 61.2 in every run.

What did not. The first lexical head barely moved from zero before early stopping. The goal-matching block (165 k parameters) was pre-registered, with its specification, acceptance rule (validation gain ≥ 1.0 point and > 2 pooled seed std) and decision procedure committed before training (experiments/goal_matching/); it lost 0.3 points on validation and was rejected without a test run. On the free T4, emulated bf16 slowed training from 108 to 377 ms per step until bf16 was restricted to GPUs that support it natively. Between run 3 and the final run, single ablations and baselines moved by up to 1.4 points, so one-point differences are noise.

Protocol, audit and evidence. The lexical head was configured on validation only, but the de-

cision to explore it was taken after the test results of runs 1 and 2 had been seen, so the small test gap to TF-IDF is indicative, not a clean held-out result. An independent audit of run 1 (docs/PROJECT_AUDIT.md) found the training/test overlap (Section 3), the order-dependent alignment (103 test items in the original pipeline, 102 now; an opt-in canonical alignment removes it), early loading of the test file and unsaved checkpoints, and added prediction export, provenance and validation checks. The submitted notebook contains the complete final run (commit 0d70cdc; 57 cells, no errors, logs of all 30 runs) and a cell that checks tables against the result files; docs/DEVELOPMENT_LOG.md records every step, including the failures.

6 Conclusion

DACT shows that task-specific structure pays off even when training from scratch: making the difference between two near-identical solutions explicit and adding direct lexical evidence yield the best from-scratch model on validation, although on test it only ties a lexical baseline; controlled ablations support the difference tags and cross-solution attention on both splits and the lexical head mainly on validation, while attention pooling and the tag bias add little. Its attention demonstrably learns to focus on the decisive words. The analysis equally shows the limit of this approach: the model finds *where* the options differ but judges *which* is right with word statistics from 14.5k items, leaving it 5–14 points behind pretrained models.

Limitations. Improvements of about one point are within seed and test-sample noise; hyperparameters were selected with one seed; the lexical head is over-confident on long differing spans; the test comparison with TF-IDF is not fully clean (Section 5); one test item overlaps with the training data; the difference tags depend on option order (102 test items change tags when the options are swapped; the opt-in canonical alignment removes this); the model overfits quickly, with training accuracy at 89–92% against about 61% on validation at the selected epoch (97–98% by the last epoch), so early stopping does most of the regularisation; and ablations that remove modules also change parameter counts. An explicit goal-matching block did not help in a pre-registered validation test (Section 5); natural next steps are

length-normalised lexical scores and combining the difference-aware design with pretrained representations.

Team Contributions

The work was divided among the three members; each implemented and tested his own part of the code and wrote the report sections that describe it.

Bayron: original DACT pipeline (data processing, model, training and evaluation code, baselines B0–B4), the original A100 run, the final A100 runs of both versions and their comparison, and integration of the final notebook and report.

Ali: model revision (difference-aware truncation, lexical head and its validation pilot), attention controls and diagnostics, three-seed RoBERTa baseline, the free-T4 runs, and the first draft of the report.

Salah: evidence audit of the original run (training/test overlap, order-dependent alignment, test exposure), robustness and provenance code (validation, determinism, prediction export), and the local replication of the original model.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439.
- Qian Chen, Xiaodan Zhu, Zhen-Hua Ling, Si Wei, Hui Jiang, and Diana Inkpen. 2017. Enhanced LSTM for natural language inference. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1657–1668.
- Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for*

- Computational Linguistics: Human Language Technologies*, pages 4171–4186.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah A. Smith. 2020. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392.
- Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel Bowman, and Noah A. Smith. 2018. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 107–112.
- Sarthak Jain and Byron C. Wallace. 2019. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3543–3556.
- Daniel Khashabi, Sewon Min, Tushar Khot, Ashish Sabharwal, Oyvind Tafjord, Peter Clark, and Hannaneh Hajishirzi. 2020. UnifiedQA: Crossing format boundaries with a single QA system. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1896–1907.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.
- Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.
- Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2249–2255.
- Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. 2018. Hypothesis only baselines in natural language inference. In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics*, pages 180–191.
- Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Qiu Ran, Peng Li, Weiwei Hu, and Jie Zhou. 2019. Option comparison network for multiple-choice reading comprehension. *arXiv preprint arXiv:1903.03033*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Sarah Wiegrefe and Yuval Pinter. 2019. Attention is not not explanation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 11–20.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, pages 10524–10533.
- Shuailiang Zhang, Hai Zhao, Yuwei Wu, Zhuosheng Zhang, Xi Zhou, and Xiang Zhou. 2020. DCMN+: Dual co-matching network for multi-choice reading comprehension. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 9563–9570.